Software Requirements Specification

Restaurant App with Server and Database

React Native, Node.js, Express and MongoDB

Version 2.0 (SRS_v2)

Mobile App Development, Fall 2026

Prepared by: Bisma

Reg. No. 9436, BS Computer Science

Abasyn University, Islamabad Campus

October 2026

1. Introduction

1.1 Purpose

This document (SRS_v2, version 2.0) revises the A1 SRS of the Restaurant App MVP. It is written for the course instructor, the developer and testers. Version 1.0 ran only on mock data stored inside each phone. Version 2.0 adds a Node.js and Express server and a MongoDB database, so every phone reads and writes the same data. Sections 1.2, 1.3, 5 and 6 are updated, Section 7 is updated to show where each screen now calls the server, and the new Section 8 lists the REST API endpoints. All other sections keep their A1 numbering and content.

1.2 Scope

In scope:

- Signup and login with a role selector (Customer or Manager), with form validation and role-based navigation.
- Menu browsing: at least 15 items in 4 categories (Starters, Mains, Desserts, Drinks), category chips, Daily Special badge, greyed-out unavailable items, pull to refresh.
- Search with debounce, recent searches, sort (price low to high, price high to low, name A to Z), back to top button and favourites.
- Cart with quantity stepper, special instructions per item, promo codes (WELCOME10 and FEAST20) and a live badge on the cart tab.
- Order summary with service charge (5%), sales tax (15%), discount and grand total; Dine-in (with table) or Takeaway (with pickup time) ordering.
- Table reservation with hourly slots from 12:00 to 22:00, availability checking, validation, confirmation modal and My Reservations with cancellation.
- Order tracking with step progress and elapsed time (status now comes from the server, polled every 10 seconds).
- Profile screen with light and dark theme toggle and logout.
- Manager Dashboard (Manager only) with Incoming Orders, Reservations (accept or decline) and Menu Management (add item, edit price, toggle availability).
- **(New in v2)** Node.js and Express REST API, MongoDB database with Mongoose, JWT authentication, password hashing, server-side validation of orders and reservations.

Out of scope:

- Real payment processing.
- Push notifications.
- Delivery tracking, maps and GPS.
- Cloud deployment of the server (it runs on the developer laptop for the demo and viva).
- State management libraries such as Redux.

The backend is no longer out of scope in v2.

Problems found in A1 and how v2 fixes them (from using the A1 app and noting where data is lost or differs between phones):

#	Problem observed in A1	How it is solved in v2
P1	Manager changes a dish price on his phone; customers still see the old price.	Menu lives in MongoDB. Menu screen calls GET /api/menu; manager edits call protected PUT /api/menu/:id. Pull to refresh shows the new price on every device.

#	Problem observed in A1	How it is solved in v2
P2	Manager marks a dish unavailable but other phones still show it as available.	isAvailable is stored once in the MenuItems collection; the server also rejects orders containing unavailable items.
P3	An order placed on one phone is not visible on another phone or to the manager's phone.	Orders are saved in the Orders collection with the user id. Customers read GET /api/orders/my; managers read GET /api/orders.
P4	Reinstalling the app deletes all orders and reservations.	Data is stored in MongoDB, not in the app. Only the login token is kept in AsyncStorage.
P5	Two phones can book the same table at the same time.	The server checks table, date, time and seats before saving and returns 409 if the table is taken.
P6	Order tracking used a fake timer, so status did not reflect what the manager did.	Order Tracking polls GET /api/orders/my every 10 seconds and shows the real status set by the manager.
P7	Passwords were plain text in a local file and anyone could act as manager.	Passwords are hashed with bcrypt; JWT tokens and the managerOnly middleware protect manager actions.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification: this document, which states what the system must do.
MVP	Minimum Viable Product: the smallest working version with the core features of the Restaurant App.
UML	Unified Modeling Language: standard diagrams (use case, class, sequence, state, component) used to design the system.
Hook	A React function (such as useState) that lets a function component use state and lifecycle features.
Context	A React mechanism (createContext and useContext) for sharing data such as the logged-in user without passing props through every level.
Reducer	A pure function (state, action) that returns the next state; used with useReducer for the cart and orders.
Mock Data	Fake sample data (users, menu, tables) written in JavaScript files and used only for the first seed of the database in v2.
FlatList	A React Native component that renders long lists efficiently by drawing only the visible rows.
Debounce	Waiting until the user stops typing (400 ms) before running the search, to avoid work on every keystroke.
AsyncStorage	A key-value store on the phone used to keep data after the app closes.
Expo / Expo Go	A toolchain and a phone app used to run and test React Native projects without building native binaries.
React Navigation	The library that provides stack and bottom-tab navigation between screens.
Dine-in / Takeaway	Dine-in: the customer eats at a restaurant table. Takeaway: the customer collects food at a chosen pickup time.
API	Application Programming Interface: the set of server URLs the app calls to get or change data.
REST	A style of API where URLs name resources (/api/menu) and HTTP methods (GET, POST, PUT, PATCH, DELETE) say what to do.
Endpoint	One URL plus HTTP method on the server, for example POST /api/orders.
JSON	JavaScript Object Notation: the text format used for request and response bodies.
MongoDB	A NoSQL document database that stores data as JSON-like documents.
Collection	A group of MongoDB documents, similar to a table (for example menuitems).
Mongoose	A Node.js library that defines schemas and models for MongoDB and validates data.
Middleware	An Express function that runs before a controller, for example protect checks the token.

Term	Definition
JWT	JSON Web Token: a signed token the server gives after login; the app sends it in the Authorization header.
bcrypt	A hashing algorithm used so passwords are never stored in readable form.
HTTP Status Code	A number in the response showing the result: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Server Error.
CORS	Cross-Origin Resource Sharing: a browser/server rule; the cors package lets the app call the server.

2. Overall Description

2.1 User Roles

Role	Key Permissions and Goals
Customer	Sign up and log in; browse, search, sort and favourite menu items; manage a cart and apply promo codes; place Dine-in or Takeaway orders; track order status; reserve a table, view and cancel own reservations; switch theme; view profile. Goal: order and book quickly without queues. Cannot open the Manager Dashboard.
Restaurant Manager	Log in with a manager account; see and update the status of incoming orders; accept or decline reservations; add menu items, edit prices and toggle availability; all customer-facing screens including theme and profile. Goal: digitise the menu and manage orders and bookings efficiently.

Self-interview (client meeting). Interviewing myself as a customer, the features that would make me download the app are: seeing today's specials without calling the restaurant; knowing which dishes are finished before I choose; searching for a dish by name; adding notes like 'no onions'; applying a discount code before paying; booking a table for 7:00 PM and being told if no table is free; and watching my order move from Pending to Served so I know when to arrive. As a manager I want to stop phone-in orders, change prices and availability in one place, and approve bookings. These points are the source of the user stories below.

2.2 User Stories

ID	User Story	Source
US-01	As a Customer, I want to see the Daily Special items, so that I can pick the best dish quickly.	Self-interview
US-02	As a Customer, I want unavailable dishes to be greyed out, so that I do not order something that is finished.	Self-interview
US-03	As a Customer, I want to search the menu by name, so that I can find a dish without scrolling.	Self-interview
US-04	As a Customer, I want to add a note such as 'no onions' to a cart item, so that my food is prepared the way I like.	Self-interview
US-05	As a Customer, I want to apply a promo code, so that I pay less.	Self-interview
US-06	As a Customer, I want to reserve a table for a chosen time and party size, so that my seat is ready when I arrive.	Self-interview
US-07	As a Customer, I want to track my order status and elapsed time, so that I know when my food will be ready.	Self-interview
US-08	As a Customer, I want to switch to dark mode, so that the app is comfortable to use at night.	Self-interview
US-09	As a Customer, I want to sort the menu by price or name, so that I can compare options.	Self-interview
US-10	As a Restaurant Manager, I want to see incoming orders and change their status, so that customers always see the real progress.	Manager
US-11	As a Restaurant Manager, I want to accept or decline reservations, so that I can control table use.	Manager

ID	User Story	Source
US-12	As a Restaurant Manager, I want to add items, edit prices and toggle availability, so that the menu is always correct on every customer's phone.	Manager
US-13	As a Restaurant Manager, I want the Manager Dashboard to be hidden from customers, so that only staff can change orders and the menu.	Manager

3. Functional Requirements

ID	Module	Requirement
FR-01	Authentication	The system shall switch the entry screen between Login mode and Signup mode.
FR-02	Authentication	The system shall require full name, email, password, confirm password and a role (Customer or Manager) for signup, and email and password for login.
FR-03	Authentication	The system shall validate on submit that the email format is valid, the password has at least 8 characters including at least one digit, and password and confirm password match.
FR-04	Authentication	The system shall clear the error of a field as soon as the user edits that field.
FR-05	Authentication	The system shall show an ActivityIndicator and disable the submit button while a request is in progress (server call).
FR-06	Authentication	The system shall open the Menu screen for a Customer and the Manager Dashboard for a Manager after successful login, and show an Alert with a clear message on failure.
FR-07	Authentication	The system shall keep the logged-in user in AuthContext and shall return to the Login screen with a reset navigation stack on logout.
FR-08	Menu	The system shall display at least 15 menu items in at least four categories (Starters, Mains, Desserts, Drinks) in a FlatList.
FR-09	Menu	The system shall show a loading indicator while the menu loads and an error message with a Retry button if loading fails.
FR-10	Menu	The system shall show a Daily Special badge on items whose isSpecial is true.
FR-11	Menu	The system shall grey out unavailable items and disable their Add to Cart button.
FR-12	Menu	The system shall filter the menu by the category chip the user selects and show the number of items in the screen header.
FR-13	Menu	The system shall reload the menu when the user pulls down to refresh.
FR-14	Search	The system shall apply the typed search text only after 400 ms without typing (debounce).
FR-15	Search	The system shall sort items by price low to high, price high to low or name A to Z.
FR-16	Search	The system shall keep the last five distinct search terms and show them as tappable suggestions when the search box is focused and empty.
FR-17	Search	The system shall show a friendly empty-state message when no item matches the search.
FR-18	Search	The system shall show a Back to top button after the user scrolls down 300 pixels.
FR-19	Cart	The system shall add an item to the cart from any menu card and show a live item count badge on the cart tab.
FR-20	Cart	The system shall increase or decrease item quantity and remove the item when its quantity reaches zero.
FR-21	Cart	The system shall let the user enter a special instruction for each cart item.
FR-22	Cart	The system shall accept the promo codes WELCOME10 (10%) and FEAST20 (20%) and reject any other code with an error message.
FR-23	Cart	The system shall calculate subtotal, 5% service charge, 15% sales tax, promo discount and grand total on the Order Summary screen.
FR-24	Reservation	The system shall offer hourly time slots from 12:00 to 22:00 and disable a slot when no table with enough seats is free at that time.

ID	Module	Requirement
FR-25	Reservation	The system shall reject a reservation whose date is in the past, whose party size is outside 1 to 12, whose phone does not match 03XX-XXXXXXX, or which starts less than one hour ahead.
FR-26	Reservation	The system shall show a confirmation modal summarising the booking before it is saved.
FR-27	Reservation	The system shall list the user's reservations and cancel one only after the user confirms an Alert.
FR-28	Orders	The system shall let the customer choose Dine-in (with a table) or Takeaway (with a pickup time) before placing an order.
FR-29	Orders	The system shall create an order with id, items, total, type, status Pending and a timestamp when the customer places it.
FR-30	Orders	The system shall show the order status as a step progress indicator with an elapsed time counter that updates every second, and shall refresh the status from the server every 10 seconds.
FR-31	Orders	The system shall allow an order to be cancelled only while its status is Pending.
FR-32	Manager Dashboard	The system shall show the Manager Dashboard tab only when user.role is manager.
FR-33	Manager Dashboard	The system shall let the manager change order status from the Incoming Orders tab.
FR-34	Manager Dashboard	The system shall let the manager accept or decline reservations from the Reservations tab.
FR-35	Manager Dashboard	The system shall let the manager add an item, edit a price and toggle availability from the Menu Management tab, and the change shall be visible to all customers after they refresh.
FR-36	Profile and Theme	The system shall show the logged-in user's name, email and role on the Profile screen.
FR-37	Profile and Theme	The system shall toggle between light and dark palettes from a switch on the Profile screen and apply it to every screen immediately.
FR-38	Server and Database (new in v2)	The system shall store users, menu items, tables, reservations and orders in MongoDB.
FR-39	Server and Database (new in v2)	The system shall store passwords only as bcrypt hashes and never return a password in any response.
FR-40	Server and Database (new in v2)	The system shall issue a JWT that expires in one day at login and require it (Bearer token) on all protected endpoints.
FR-41	Server and Database (new in v2)	The system shall return 401 when no valid token is sent and 403 when a Customer calls a manager-only endpoint.
FR-42	Server and Database (new in v2)	The system shall calculate the order total on the server from database prices and shall reject items that are unavailable.
FR-43	Server and Database (new in v2)	The system shall allow order status to move only forward (Pending, Preparing, Ready, Served) and allow Cancelled only from Pending, otherwise return 400.
FR-44	Server and Database (new in v2)	The system shall return 409 when a reservation is requested for a table that is already booked at that date and time or has too few seats.
FR-45	Server and Database (new in v2)	The system shall return 400 for invalid or missing data and for an invalid MongoDB id, and 404 when a record does not exist.

ID	Module	Requirement
FR-46	Server and Database (new in v2)	The system shall keep the user logged in after the app is closed by saving the token and user in AsyncStorage, and clear both on logout.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	Every screen shall use the same navigation pattern (bottom tabs plus stacks), large touch targets (minimum 44 px) and clear inline error messages. A new user shall be able to place an order in under 2 minutes without help.
NFR-02	Usability	Both light and dark themes shall keep text readable (sufficient contrast) on every screen.
NFR-03	Performance	A screen shall show its first content or a loading indicator within 1 second; the menu list (15+ items) shall scroll smoothly using FlatList with keyExtractor.
NFR-04	Performance	MenuItemCard shall be wrapped in React.memo with useCallback handlers so that toggling one favourite re-renders only that card; totals shall be computed with useMemo.
NFR-05	Responsiveness	Layouts shall adapt to small (360 px wide) and large phones and tablets using flex layouts and percentages, and shall respect safe areas (notch and home indicator).
NFR-06	Maintainability	Source shall be organised under src with the folders components, screens, context, reducers, hooks, data, navigation and api; business logic shall live in hooks and reducers, not in screens.
NFR-07	Maintainability	Reusable components (MenuItemCard, CategoryChips, SearchBar, Loader) shall be reused instead of copied; colours shall come only from the theme file.
NFR-08	Data handling	All business data shall live in MongoDB. AsyncStorage shall hold only the token and user. The A1 mock menu and users files shall be removed from the app.
NFR-09	Security	Passwords shall be hashed with bcrypt, JWT_SECRET and MONGO_URI shall be kept in .env (never committed), and .env.example shall be committed with empty values.
NFR-10	Reliability	The server shall not crash on bad input; every controller shall use try and catch and the server shall return JSON errors (404 Route not found, 500 handler).
NFR-11	Performance	API responses for menu, orders and reservations shall return within 2 seconds on the local network.
NFR-12	Portability	BASE_URL shall be defined in one place (src/api/client.js); the Android emulator uses http://10.0.2.2:5000 and a phone uses the laptop IP on the same Wi-Fi.

5. Database Design (replaces A1 Section 5)

The A1 mock data sets become MongoDB collections in the database **restaurant_app**, accessed through Mongoose models. Every document also has an automatic _id (ObjectId).

Collection	Fields and Types	Relationships and Rules
users	name: String (required); email: String (required, unique, lowercase); password: String (required, bcrypt hash, min length before hashing 8); role: String enum ['customer','manager'], default 'customer'	Email unique. Password never returned by the API. Referenced by orders.user and reservations.user.
menuitems	name: String (required); description: String; price: Number (required, min 0); category: String enum ['Starters','Mains','Desserts','Drinks'] (required); image: String; isSpecial: Boolean, default false; isAvailable: Boolean, default true	Referenced by orders.items.menuItem. Price is always read from here when an order is priced.
tables	number: Number (required, unique); seats: Number (required, min 1)	Referenced by reservations.table and orders.table. At least six seeded tables.

Collection	Fields and Types	Relationships and Rules
reservations (timestamps)	user: ObjectId ref users (required); table: ObjectId ref tables (required); date: Date (required); timeSlot: String (required, '12:00' to '22:00'); partySize: Number (required, 1 to 12); phone: String (required, 03XX-XXXXXXX); status: String enum ['Pending','Accepted','Declined','Cancelled'], default 'Pending'; createdAt, updatedAt	Each reservation stores the id of the user and the table. No two active reservations may have the same table, date and timeSlot (checked in the controller, 409).
orders (timestamps)	user: ObjectId ref users (required); items: [{ menuItem: ObjectId ref menuitems, name: String, price: Number, quantity: Number (min 1), note: String }]; promoCode: String; discountPercent: Number; total: Number (computed by the server); type: String enum ['Dine-in','Takeaway']; table: ObjectId ref tables (Dine-in only); pickupTime: String (Takeaway only); status: String enum ['Pending','Preparing','Ready','Served','Cancelled'], default 'Pending'; createdAt, updatedAt	Each order stores the id of the user who placed it. Item name and price are copied at order time so later price changes do not alter old orders. Status may only move forward; Cancelled only from Pending.

Promo codes (WELCOME10, FEAST20) remain a small constant object in the server order controller.

6. UML Diagrams and System Diagrams

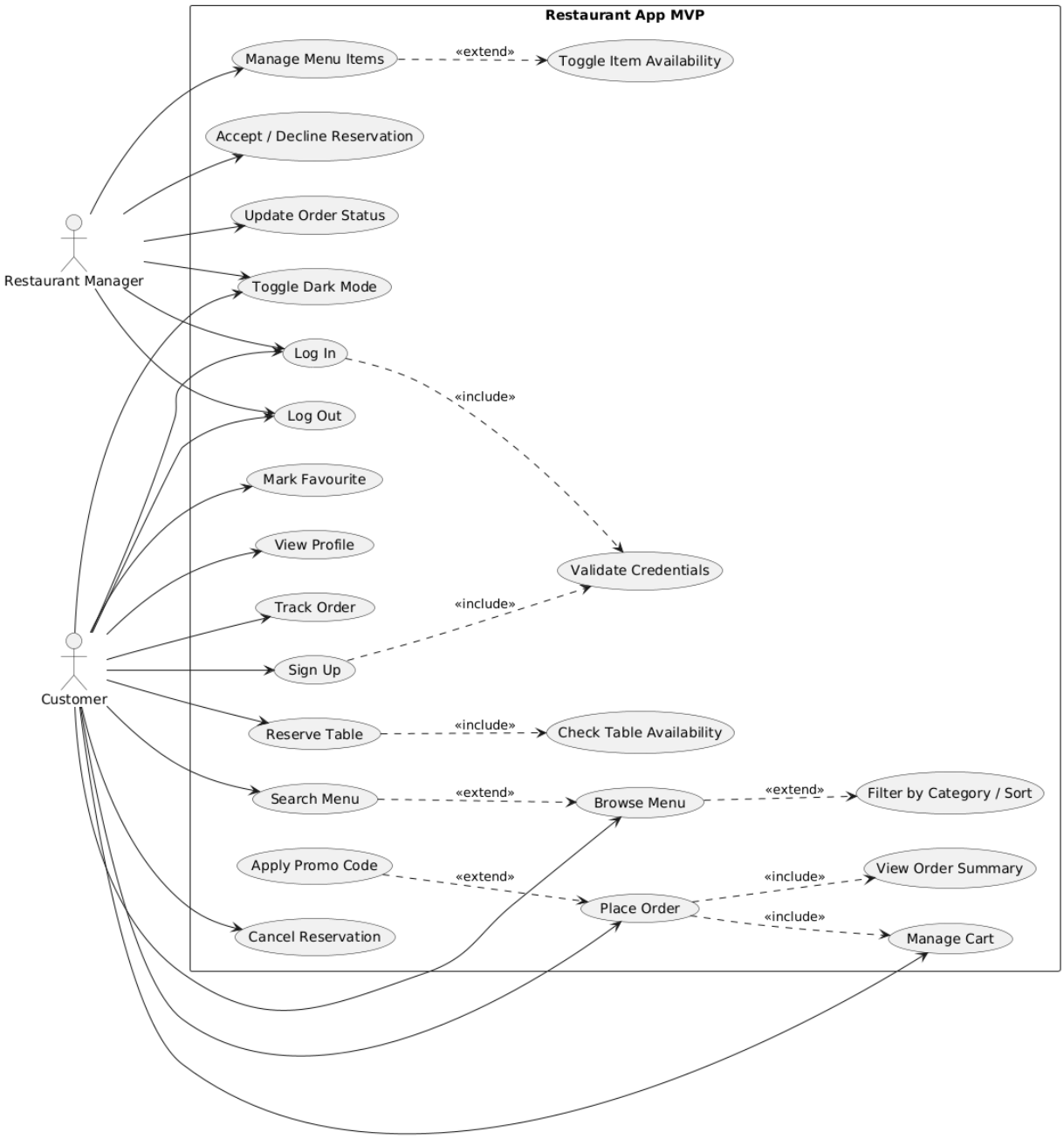


Figure 6.1: Use Case Diagram

The use case diagram shows the two actors, Customer and Restaurant Manager, and 22 use cases. Place Order includes View Order Summary and Manage Cart, Sign Up and Log In include Validate Credentials, and Reserve Table includes Check Table Availability. Apply Promo Code extends Place Order, Search Menu and Filter/Sort extend Browse Menu, and Toggle Item Availability extends Manage Menu Items. Both actors share Log In, Toggle Dark Mode and Log Out.

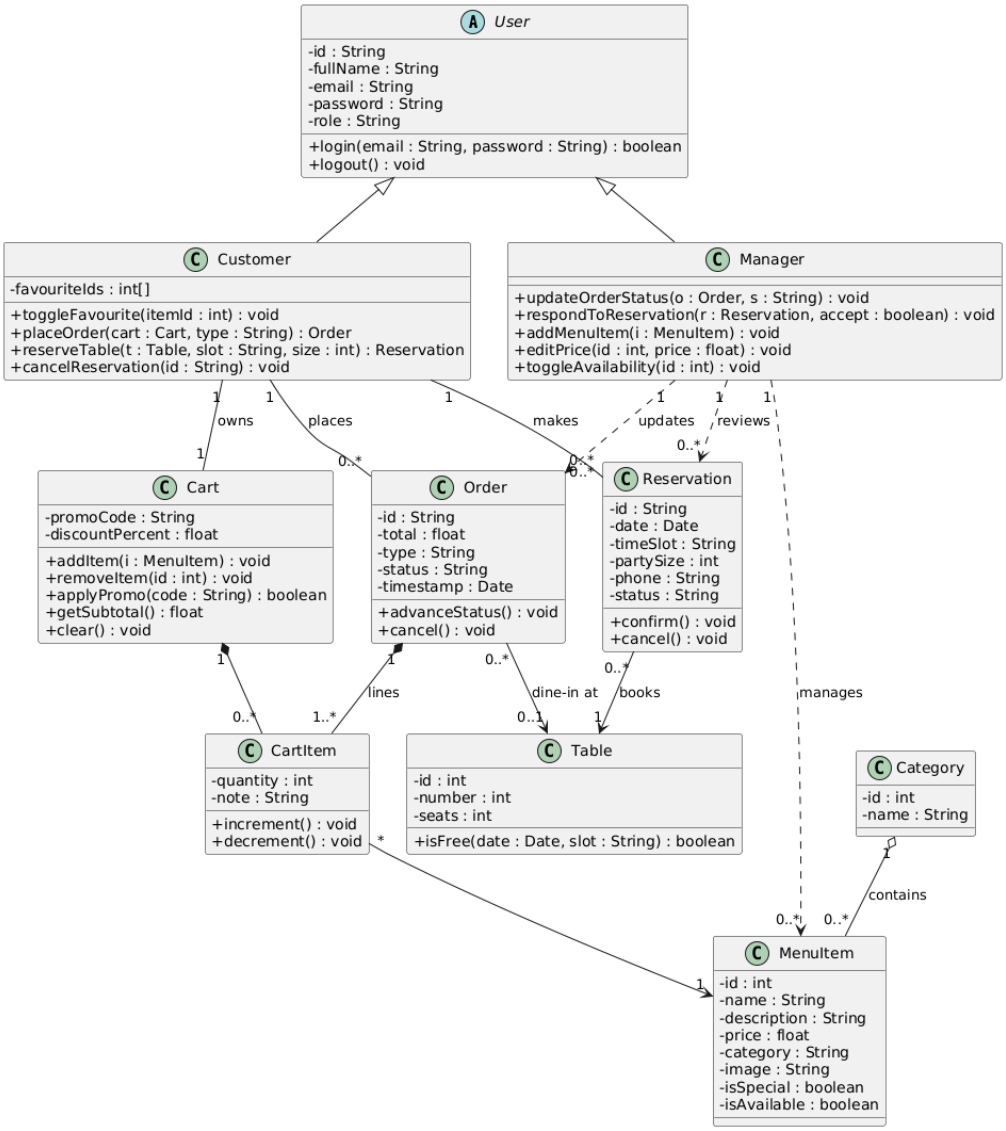


Figure 6.2: Class Diagram

The class diagram shows User as a parent class of Customer and Manager (inheritance). A Customer owns one Cart made of CartItems that point to MenuItems, places many Orders and makes many Reservations; each Reservation books one Table. Category groups MenuItems. Attributes show types and multiplicities are written on the lines. Managers update Orders, review Reservations and manage MenuItems.

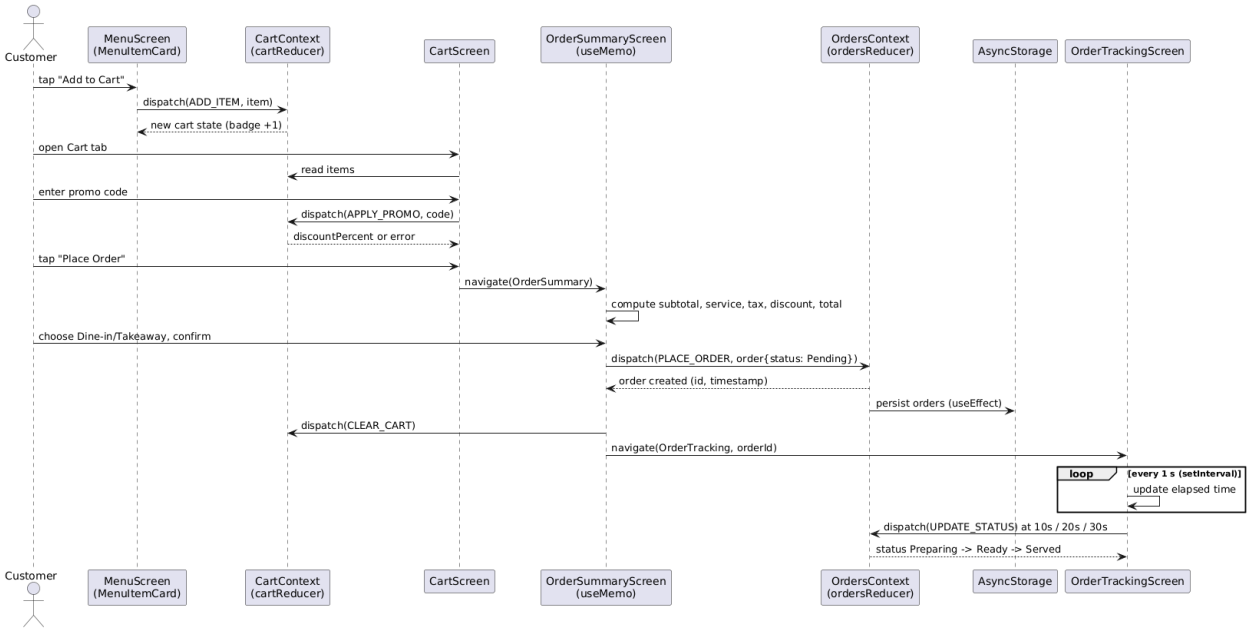


Figure 6.3: Sequence Diagram - Customer places an order (frontend flow; see Figure 6.6 for the server version)

The sequence diagram follows the scenario from tapping Add to Cart on a menu card to seeing the Order Tracking screen. The cart reducer handles ADD_ITEM and APPLY_PROMO, the Order Summary screen computes totals with useMemo, OrdersContext creates the order with status Pending and saves it to AsyncStorage, the cart is cleared, and Order Tracking runs a one second timer and advances the status.

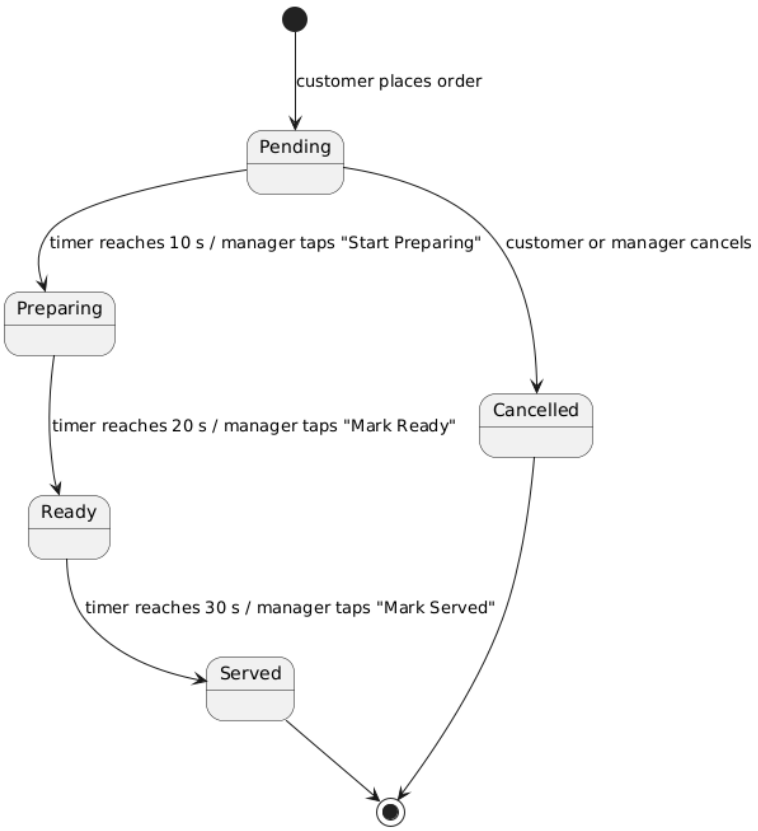


Figure 6.4: State Machine Diagram - Order lifecycle

An order starts in Pending when placed. It moves to Preparing, then Ready, then Served, either automatically (demo timers at 10, 20 and 30 seconds) or when the manager changes it. Cancellation is allowed only from Pending. Served and Cancelled are final states.

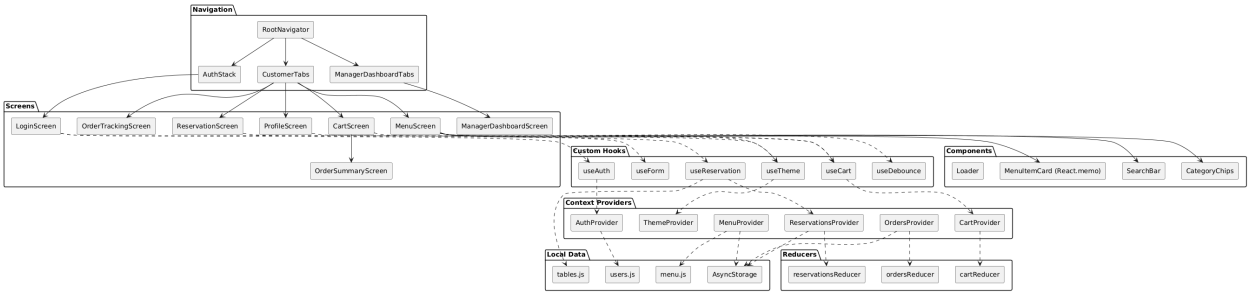


Figure 6.5: Component Diagram - React Native app

The component diagram shows the navigators, the screens, the reusable components, the context providers with their reducers, the custom hooks and the data layer. Screens depend on hooks, hooks depend on providers, providers depend on reducers and on local data or AsyncStorage.

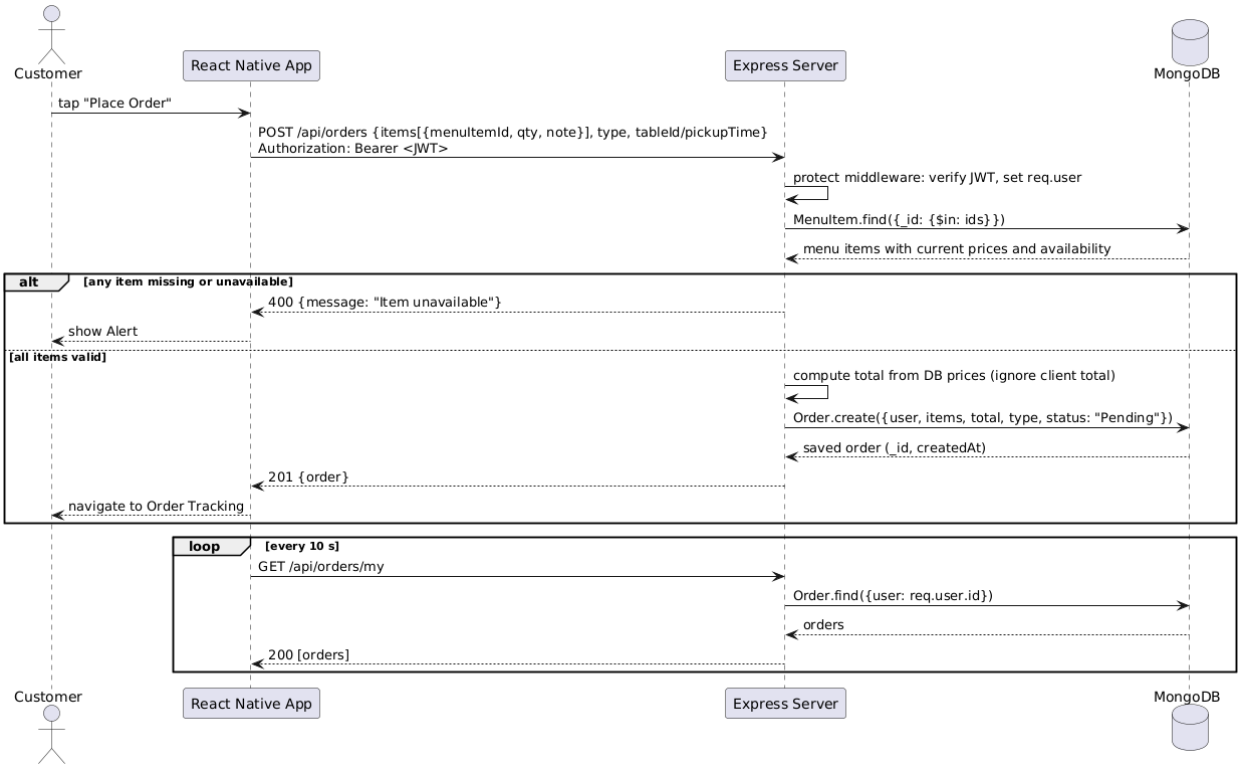


Figure 6.6: Sequence Diagram - Customer places an order (App, Express Server, MongoDB)

The updated sequence diagram has three lifelines. The app sends POST /api/orders with the JWT. The protect middleware verifies the token, the server reads the current prices and availability from MongoDB, rejects the request with 400 if an item is unavailable, otherwise calculates the total itself, saves the order and returns 201. The app then polls GET /api/orders/my every 10 seconds.

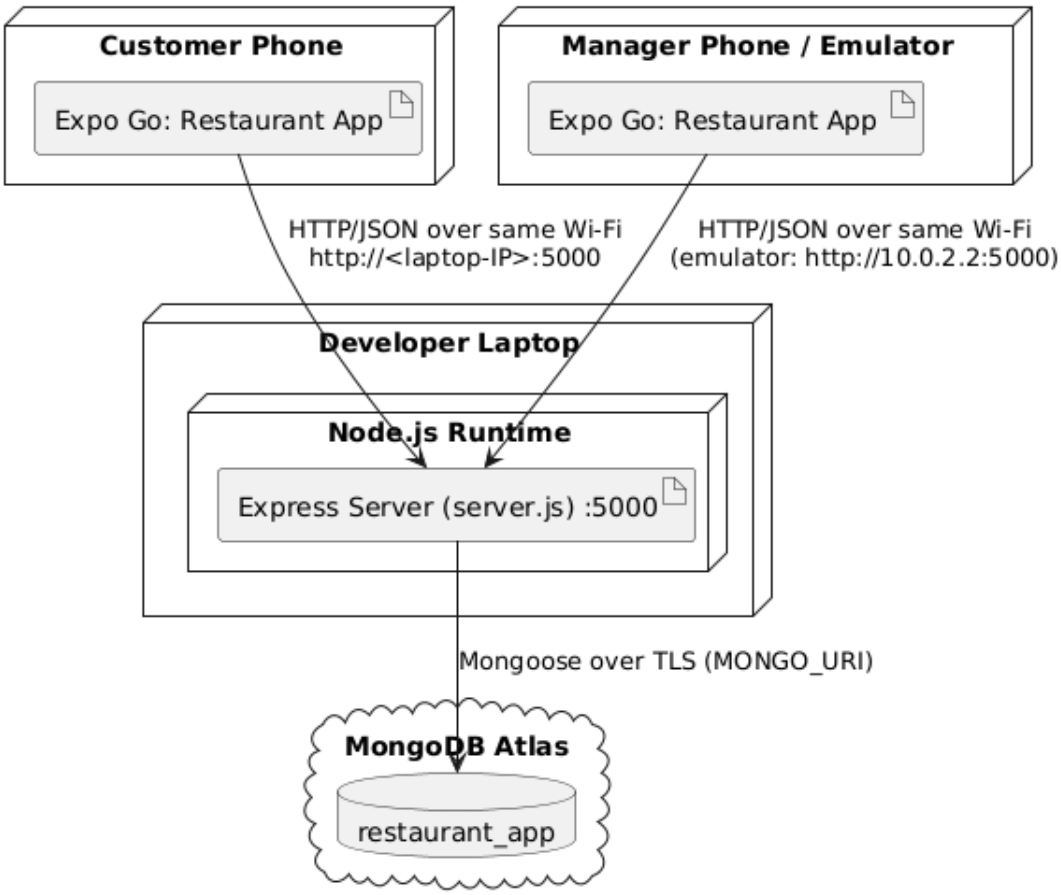


Figure 6.7: Deployment Diagram

The customer phone and the manager phone (or emulator) run the Expo app and talk over HTTP on the same Wi-Fi to the Express server running on the developer laptop at port 5000. The server connects to the restaurant_app database on MongoDB Atlas using Mongoose.

7. MVP Frontend Development (React Native)

7.1 Login and Signup Screen

Purpose	Let a user create an account or log in and route them by role.
UI elements	Mode switch (Login/Signup), TextInputs for full name, email, password, confirm password, role selector (Customer/Manager), show-password toggle, error text under each field, submit button with ActivityIndicator, Alert on failure.
Navigation (entry / exit)	Entry: app start when no user is logged in; also after logout. Exit: Customer goes to Menu tab; Manager goes to Manager Dashboard.
Data used	Server: POST /api/auth/register and POST /api/auth/login; token and user saved to AuthContext and AsyncStorage.
Hooks and why	useState (mode, values, errors, showPassword, isSubmitting) for form UI state; useForm custom hook (values, errors, handleChange, handleSubmit, reset, isValid) to keep validation out of the screen; useAuth (useContext) to store the user; useApi/client helper for the request.

7.2 Menu Browsing Screen

Purpose	Show the menu, filter by category and add items to the cart.
UI elements	Horizontal category chips, FlatList of MenuItemCards (image, name, price, Daily Special badge, favourite heart, Add button), loading indicator, error message with Retry, pull to refresh, header with item count.
Navigation (entry / exit)	Entry: Menu tab (default for Customer). Exit: Cart tab, Search controls on the same screen.
Data used	Server: GET /api/menu (optional category and search); favourites kept in state.
Hooks and why	useState (menuItems, isLoading, error, selectedCategory, refreshing, favouriteIds); useEffect with empty dependency array to load data with cleanup that clears the timer; useMemo for filtered and sorted list (derived data is not stored in state); useCallback + React.memo for MenuItemCard so only the toggled card re-renders; useApi returns data, loading, error, refetch.

7.3 Search and Scroll Controls

Purpose	Let the user find dishes fast and move around a long list.
UI elements	Search bar with search icon and clear button, recent-search suggestions (last five), sort selector, empty-state message, floating Back to top button, render counter debug label.
Navigation (entry / exit)	Entry: part of the Menu screen. Exit: none (same screen).
Data used	Menu items in state; recentSearches array in state.
Hooks and why	useRef for the TextInput (focus), the FlatList (scrollToOffset) and the render counter (changing a ref does not re-render, changing state does); useDebounce(value, 400) custom hook for the search text; useState for scroll position (button visible after 300 px) and recent searches; useMemo for filtering and sorting.

7.4 Profile and Theme Screen

Purpose	Show account details and let the user switch between light and dark theme or log out.
UI elements	Name, email and role fields, dark mode Switch, Logout button.
Navigation (entry / exit)	Entry: Profile tab. Exit: Login screen on logout (navigation stack reset).
Data used	AuthContext user; theme palettes in one theme file.
Hooks and why	useContext through useAuth and useTheme (both throw an error outside their provider) so every screen reads user and colours without prop drilling.

7.5 Cart Screen

Purpose	Review and change the cart, apply a promo code.
UI elements	List of cart items with quantity stepper, remove button, special-instructions TextInput, promo code field with Apply and Remove, subtotal, Place Order button, tab badge with item count.
Navigation (entry / exit)	Entry: Cart tab. Exit: Order Summary screen.
Data used	Cart state { items, promoCode, discountPercent }; local promo object (WELCOME10, FEAST20); promo is also checked by the server when the order is placed.
Hooks and why	useReducer with cartReducer (ADD_ITEM, REMOVE_ITEM, INCREMENT, DECREMENT, UPDATE_NOTE, CLEAR_CART, APPLY_PROMO, REMOVE_PROMO) because the cart has several related transitions; CartContext and useCart share it with the menu cards and tab badge.

7.6 Order Summary Screen

Purpose	Show the price breakdown and confirm the order.
UI elements	Line items, subtotal, service charge 5%, sales tax 15%, promo discount, grand total, Dine-in/Takeaway selector, table picker or pickup time, Confirm button.
Navigation (entry / exit)	Entry: from Cart. Exit: Order Tracking screen after the order is placed.
Data used	Cart items and discount; constants SERVICE_RATE = 0.05 and TAX_RATE = 0.15; the server recalculates the final total from database prices.
Hooks and why	useMemo for totals depending only on cart items and discount; useState for order type, table and pickup time; useContext (OrdersContext) to dispatch the new order.

7.7 Table Reservation Screen

Purpose	Book a table for a date, time and party size.
UI elements	Date picker, hourly time-slot chips (12:00 to 22:00, disabled when unavailable), party size, table choice, phone input, confirmation modal, My Reservations list with Cancel.
Navigation (entry / exit)	Entry: Reservations tab. Exit: stays on screen; cancel shows an Alert.
Data used	Server: POST /api/reservations, GET /api/reservations/my, PATCH cancel; server returns 409 if the table is taken.
Hooks and why	useReservation custom hook holds all business logic (selection state, availability check, createReservation, cancelReservation) so the screen has only UI code; useForm for validation; useState for modal visibility.

7.8 Order Tracking and Manager Dashboard

Purpose	Customer: watch the order progress. Manager: manage orders, reservations and the menu.
UI elements	Tracking: step progress indicator, elapsed time counter, status text. Dashboard: three tabs (Incoming Orders with status buttons, Reservations with Accept/Decline, Menu Management with add item, edit price, availability switch).
Navigation (entry / exit)	Tracking entry: after placing an order. Dashboard entry: Manager tab, visible only when user.role is manager.
Data used	Server: GET /api/orders/my (polled every 10 s), GET /api/orders, PATCH /api/orders/:id/status, GET /api/reservations, PATCH /api/reservations/:id, POST/PUT /api/menu.
Hooks and why	useEffect with setInterval for the elapsed timer and the 10 second polling, cleared in cleanup; useApi for loading lists; useContext for user and theme; token read from AuthContext for protected calls.

8. API Endpoints

Base URL: http://<laptop-IP>:5000 (Android emulator: http://10.0.2.2:5000). Protected endpoints need the header Authorization: Bearer <token>. Errors return JSON { message }.

Method	Endpoint	Access	Request Body	Response
POST	/api/auth/login	Public	email, password	token and user
GET	/api/menu	Public	None	List of menu items
GET	/api/health	Public	None	{ status: 'ok', time }
POST	/api/auth/register	Public	name, email, password, role	201 token and user; 409 if email exists
GET	/api/menu?category=Mains&search;=burger	Public	None (query params optional)	Filtered list of menu items
GET	/api/menu/:id	Public	None	One menu item; 404 if not found; 400 if invalid id
POST	/api/menu	Manager (protect + managerOnly)	name, description, price, category, image, isSpecial, isAvailable	201 created item; 400 on bad data
PUT	/api/menu/:id	Manager	Any menu fields to update (for example price, isAvailable)	200 updated item; 404 if not found
DELETE	/api/menu/:id	Manager	None	200 message; 404 if not found
GET	/api/tables	Protected	None	List of tables with seats
POST	/api/orders	Customer (protect)	items [{menuItem, quantity, note}], type, table or pickupTime, promoCode	201 created order with server-calculated total; 400 if item unavailable
GET	/api/orders/my	Customer (protect)	None	Orders of the logged-in user
GET	/api/orders	Manager	None	All orders
PATCH	/api/orders/:id/status	Manager (Customer may only cancel own Pending order)	status	200 updated order; 400 if the change is not allowed
POST	/api/reservations	Customer (protect)	table, date, timeSlot, partySize, phone	201 reservation; 409 if table booked; 400 if invalid
GET	/api/reservations/my	Customer (protect)	None	Reservations of the logged-in user
GET	/api/reservations	Manager	None	All reservations
PATCH	/api/reservations/:id	Manager (Customer may cancel own)	status (Accepted, Declined or Cancelled)	200 updated reservation
GET	/api/reservations/availability?date=&timeSlot;=&partySize;=	Protected	None (query params)	Tables free at that time with enough seats (used to disable slots)
GET	/api/auth/me	Protected	None	Current user (without password)

Status codes used: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized (no or bad token), 403 Forbidden (not a manager), 404 Not Found, 409 Conflict, 500 Server Error.